This architecture is designed for an **AI-powered Hospital Management System**, prioritizing **HIPAA/GDPR compliance**, high availability, and secure data handling.

---

1. Deployment Architecture

We will use a **Microservices Architecture** deployed on **Kubernetes (EKS/GKE)** to isolate sensitive AI workloads from core CRUD hospital operations.

- **Ingress:** Nginx Ingress Controller with TLS termination.
- **API Gateway:** Kong or Traefik to handle rate limiting and authentication.
- **Network:** Multi-zone VPC with private subnets for DB and AI model services.
- **Security:** WAF (Web Application Firewall) at the Edge and mTLS between services.

---

2. Docker

- **Base Images:** Use `python:3.11-slim` (for AI) and `node:20-alpine` (for Backend) to minimize attack surface.
- **Multi-stage builds:** Separate build-time dependencies from runtime environments.
- **Security:** Integrate **Trivy** in the CI pipeline to scan images for vulnerabilities.

---

3. CI/CD (GitHub Actions / GitLab CI)

- **Stages:**

1. **Lint/Test:** Unit tests + Jest/PyTest.
2. **Security Scan:** SAST (SonarQube) + Container Scanning.
3. **Build/Push:** Push image to ECR/GCR with immutable tags.
4. **Deploy:** Helm chart deployment via ArgoCD (GitOps approach).

---

4. Hosting

- **Cloud Provider:** AWS (Recommended for specialized AI hardware like Inferentia/Trainium).
- **Infrastructure:** Kubernetes (EKS) for orchestration.
- **AI Inference:** AWS SageMaker or dedicated GPU nodes for heavy LLM/Computer Vision model inference.

---

5. Database Hosting

- **Primary DB:** AWS RDS (PostgreSQL) with **Multi-AZ failover**. (Supports ACID transactions for patient records).
- **Caching:** Redis (ElastiCache) for session management and fast retrieval of doctor schedules.
- **Vector DB:** Pinecone or Milvus (for AI-powered patient history search/semantic retrieval).

---

6. Environment Variables

- **Secrets Management:** HashiCorp Vault or AWS Secrets Manager.
- **Injection:** Kubernetes Secrets/ConfigMaps (mounted as files or env vars).
- **Examples:**

* `DB_CONN_STR`, `REDIS_URL`

* `JWT_SECRET_KEY`, `ENCRYPTION_KEY_AT_REST`

* `AI_MODEL_ENDPOINT`, `API_RATE_LIMIT_TIER`

---

7. Monitoring

- **Metrics:** Prometheus + Grafana (Monitoring API latency, request error rates, and GPU utilization).
- **Tracing:** Jaeger or AWS X-Ray (Visualizing the path of a request from UI through the AI analysis pipeline).
- **Alerting:** PagerDuty integration for critical service failures.

---

8. Logging

- **Stack:** EFK (Elasticsearch, Fluentd, Kibana) or CloudWatch Logs.
- **Compliance:** Mask PII (Personally Identifiable Information) before log ingestion.
- **Retention:** 90-day hot storage, 7-year cold storage (S3 Glacier) for regulatory compliance.

---

9. Scaling

- **Horizontal Pod Autoscaler (HPA):** Scale API replicas based on CPU/Memory usage.
- **Cluster Autoscaler:** Automatically add new EC2 nodes to the EKS cluster during peak hospital admission hours.
- **Database:** Read replicas for heavy reporting queries, independent of the primary write node.

---

10. Backup Strategy

- **Database:** RDS Point-in-Time Recovery (PITR) with daily snapshots kept for 30 days.
- **Application Data:** S3 Versioning for document storage (patient records, prescriptions).
- **Disaster Recovery:** Cross-region replication of backups to a secondary AWS region to meet healthcare regulatory requirements for data durability.

Architect's Pro-Tip for Healthcare:

Ensure **Encryption at Rest** (AES-256) and **Encryption in Transit** (TLS 1.3) are mandatory. For the AI component, ensure that the model inference container does not store patient data locally; use an ephemeral volatile memory store if necessary to avoid accidental data persistence on the node disk.